

Blockview Sol Keys

Smart Contract Security Assessment

March 2025

Prepared for:

Blockview.ai

Prepared by:

Offside Labs

Yao Li

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	4
4	Key Findings and Recommendations	5
4.1	Direct Token Burning Will Cause Supply Accounting Mismatch	5
4.2	Improper Empty Account Check May Cause DoS Attack in buy IX	6
4.3	Possible Sandwich or Front Running Attack in sell_sol_key IX	7
4.4	Improper Edge Case Handling for Bonding Curve Calculation	8
4.5	Short Key Suffix in Metadata Allows Malicious Key Collision	8
4.6	Informational and Undetermined Issues	9
5	Disclaimer	11



1 About Offside Labs

Offside Labs stands as a pre-eminent security research team, comprising highly skilled hackers with top - tier talent from both academia and industry.

The team demonstrates extensive and diverse expertise in modern software systems, which encompasses yet are not restricted to *browsers, operating systems, IoT devices, and hypervisors*. Offside Labs is at the forefront of innovative domains such as *cryptocurrencies and blockchain technologies*. The team achieved notable accomplishments including the successful execution of remote jailbreaks on devices like the **iPhone** and **PlayStation 4**, as well as the identification and resolution of critical vulnerabilities within the **Tron Network**.

Offside Labs actively involves in and keeps contributing to the security community. The team was the winner and co-organizer for the *DEFCON CTF*, the most renowned CTF competition in Web2. The team also triumphed in the **Paradigm CTF 2023** in Web3. Meanwhile, the team has been conducting responsible disclosure of numerous vulnerabilities to leading technology companies, including *Apple, Google, and Microsoft*, safeguarding digital assets with an estimated value exceeding **\$300 million**.

During the transition to Web3, Offside Labs has attained remarkable success. The team has earned over **\$9 million** in bug bounties, and **three** of its innovative techniques were acknowledged as being among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



2 Executive Summary

Introduction

Offside Labs completed a security audit of *SolKeys* smart contracts, starting on February 20, 2025, and concluding on February 25, 2025.

Project Overview

The Sol-Keys project implements a social token system on Solana where users can mint and trade “keys” (tokens) associated with their identity. The price of these keys follows a quadratic bonding curve, creating a unique economic model where early supporters pay less and prices increase with supply.

Audit Scope

The assessment scope contains mainly the smart contracts of the sol-keys program for the *Blockview* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Blockview-sol-keys
 - Codebase: <https://github.com/Blockview-ai/blockview-sol-keys>
 - Commit Hash: 3ee2e72d3e4489fd2d8b606058759fa55cd90523

We listed the files we have audited below:

- Blockview-sol-keys
 - program/src/**/*rs

Findings

The security audit revealed:

- 0 critical issue
- 1 high issues
- 2 medium issues
- 2 low issues
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Direct Token Burning Will Cause Supply Accounting Mismatch	High	Fixed
02	Improper Empty Account Check May Cause DoS Attack in buy IX	Medium	Fixed
03	Possible Sandwich or Front Running Attack in sell_sol_key IX	Medium	Fixed
04	Improper Edge Case Handling for Bonding Curve Calculation	Low	Fixed
05	Short Key Suffix in Metadata Allows Malicious Key Collision	Low	Fixed
06	Missing Validation on fee_basis_point and pubkey in create and update IXs	Informational	Acknowledged
07	Inconsistent FeeCollector Reallocation Logic Causes Excessive Space Allocation	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Direct Token Burning Will Cause Supply Accounting Mismatch

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `sell` instruction, the following check is performed to prevent the creator from selling their last token:

```
105 // Prevent the original authority from selling their last key
106 if ctx.accounts.seller.key() == ctx.accounts.original_authority.key()
107 {
108     let seller_token_amount = ctx.accounts.token_account.amount;
109     require_gt!(
110         seller_token_amount.checked_sub(args.amount).unwrap_or(0),
111         0,
112         SolKeysError::CantSellLastKey
113     );
114 }
```

[program/src/instructions/sol_keys/sell.rs#L105-L113](#)

It infers the token supply will never become zero. However, the creator can burn the tokens through token program directly to break this invariant.

Additionally, the direct token burning will also cause the discrepancy between the supply and the cost accumulated in the Native Pool.

Impact

1. Invariant Violation: The creator can burn tokens directly, reducing the total supply to zero, which breaks the assumption that a minimum supply always exists.
2. Native Pool Exploit: If the supply becomes zero, we have the following pricing calculations:

- `calculate_price_many_checked(0, 1, Add) = 0`
- `calculate_price_many_checked(1, 1, Remove) = curve.base`

An attacker can transfer `curve.base - rent` lamports to the Native Pool account and then perform a buy and a sell instructions with an amount of 1. This will result in the Native Pool account being closed, as its balance will be reduced to zero.

3. Supply-Cost Mismatch: If any user burns tokens directly, the token supply and the accumulated cost in the Native Pool will no longer align. This results in SOL being stuck in the Native Pool account, with no mechanism to recover it.



Recommendation

1. Maintain an internal record of the token supply within the program itself, rather than relying solely on the token program's supply.
2. Implement a dedicated instruction to allow the recovery of stuck SOL from the Native Pool account.

Mitigation Review Log

Fixed in the commit [682d20b82f908c8790660262bfa6ff6db6ba063a6](#).

4.2 Improper Empty Account Check May Cause DoS Attack in buy IX

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: DoS Attack

Description

In the `buy` instruction, the following logic is used to determine if an Associated Token Account (ATA) should be created:

```
141     if is_account_empty(token_account) {  
142         ctx.accounts  
143             .init_token_account(token_account,  
144     &ctx.accounts.buyer.to_account_info());  
145     }
```

[program/src/instructions/sol_keys/buy.rs#L141-L144](#)

The `is_account_empty` function checks whether an account is “empty” as follows:

```
13  /// Checks if the account is empty by comparing the owner to the system  
14  & program  
15  /// and checking if the data is empty along it's lamports.  
16  pub fn is_account_empty(account: &AccountInfo) -> bool {  
17      account.owner.key() == system_program::ID && account.data_is_empty()  
18      && account.lamports() == 0  
19  }
```

[program/src/magic/assertions.rs#L13-L17](#)

However, this logic contains a vulnerability in the balance check. Specifically, since the address of the ATA token account is deterministic, anyone can send lamports to the uninitialized account before the ATA is created. This will cause `is_account_empty` to return `false`, which skips the account initialization step. Consequently, the `buy` instruction will



fail when attempting to interact with the uninitialized ATA, resulting in a Denial of Service (DoS) attack.

Impact

An attacker can exploit this vulnerability by transferring a small amount of lamports to a deterministically calculated uninitialized ATA account, preventing its initialization and causing the `buy` instruction to fail.

Recommendation

Remove the balance check `account.lamports() == 0` in the `is_account_empty` function.

Mitigation Review Log

Fixed in the commit [682d20b82f908c8790660262bfaff6db6ba063a6](#).

4.3 Possible Sandwich or Front Running Attack in `sell_sol_key` IX

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: MEV Risk

Description

In the `buy_sol_key` IX, a `max_price` limit is applied to the purchase price.

However, in the `sell_sol_key` IX, there is no protection for the selling price. This may cause users to sell their SOL key at a lower price than expected.

Impact

Arbitragers can exploit this by performing a sandwich or front running attack on the sell tx, leading to the user selling their SOL key at an even lower price than anticipated. The lost amount is then captured by the arbitragers.

Recommendation

It is recommended to add a `min_price` limit to restrict the selling price and prevent users from suffering unexpected losses.

Mitigation Review Log

Fixed in the commit [682d20b82f908c8790660262bfaff6db6ba063a6](#).



4.4 Improper Edge Case Handling for Bonding Curve Calculation

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The bonding curve calculation contains improper handling of edge cases, potentially leading to numerical overflow and inconsistencies in value calculation. Specifically, when `n > 1`, calling `calculate_price_many_checked(n, n, Remove)` will fail due to a numerical overflow error caused by supply shifting with -2.

Additionally, there is a discrepancy in the results of `calculate_price_many_checked`. For example, `calculate_price_many_checked(0, 1, Add)` returns `0`, while `calculate_price_many_checked(1, 1, Remove)` returns `curve.base`. This inconsistency creates an irregularity in the bonding curve's price calculations and can potentially be leveraged to exploit other issues, such as the "Direct Token Burning Will Cause Supply Accounting Mismatch" vulnerability.

Impact

These edge cases cannot be directly exploited but can amplify the impact of other vulnerabilities.

Recommendation

Improve the calculation for these edge cases or add explicit checks to return errors for them.

Mitigation Review Log

Fixed in the commit [682d20b82f908c8790660262bfaff6db6ba063a6](#).

4.5 Short Key Suffix in Metadata Allows Malicious Key Collision

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Collision

Description

In the `mint_sol_key` IX, the program initializes Metadata for the newly created Mint. The name, symbol, and URI in the metadata are generated from the authority's base58 pubkey. The `key_suffix` here is derived from 6 characters of the pubkey string.



```
165 let key_suffix =
166 self.authority.key().to_string().chars().take(6).collect::<String>();
167
168 let name = format!("Blockview Key {}", key_suffix);
169 let symbol = format!("BVKEY{}", key_suffix);
170 let uri = format!(
171     "https://assets.blockview.ai/metadata/solkey/{}",
172     self.authority.key()
173 );
```

[program/src/instructions/sol_keys/mint.rs#L165-L172](#)

A malicious user can generate another pubkey with a different value but the same `key_suffix` within a few hours, creating a malicious authority keypair.

Impact

For a target Blockview Key A, a malicious user can quickly generate another malicious authority and create another malicious Blockview Key B, making A and B have same name, symbol, and URI. As a result, users intending to interact with A may be misled to B due to the identical name, symbol, and URI.

Recommendation

It is recommended that the `key_suffix` length be at least 12 to 15 characters to significantly reduce the probability of authority key suffix collisions.

Mitigation Review Log

Fixed in the commit [682d20b82f908c8790660262bfaff6db6ba063a6](#).

4.6 Informational and Undetermined Issues

Missing Validation on `fee_basis_point` and `pubkey` in `create` and `update` IXs

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Data Validation

When `create` and `update` the `fee_collector`, there is no check to ensure that the `buy_fee_basis_points` and `sell_fee_basis_points` of the `FeeCollector` are less than 10,000.

If the `protocol_instance` authority mistakenly sets these values above 10,000, unexpected fees may be charged. Additionally, when multiple `FeeCollectors` exist, it is recommended to check whether the total `fee_basis_points` of all `FeeCollectors` combined is also less than 10,000.



Additionally, the program does not check for duplicates within the provided `fee_collectors`, allowing multiple `fee_collectors` to be created for the same pubkey.

Inconsistent FeeCollector Reallocation Logic Causes Excessive Space Allocation

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

In the `update` IX, the space size for account reallocation is calculated based on the number of `fee_collectors` and `remove_fee_collector` in the input parameters. The `fee_collector` is then inserted into the `protocol_instance`, and unnecessary `fee_collectors` are removed.

However, `calc_fee_collectors_len` calculates the required space by first deleting and then inserting `fee_collectors`, whereas in `protocol_instance::update::processor`, the operation order for `fee_collectors` in `protocol_instance` is first upsert and then delete. If the same `fee_collector` appears in both `UpdateProtocolInstanceArgs.fee_collectors` and `UpdateProtocolInstanceArgs.remove_fee_collector`, this may result in reallocating excess space and paying more rent for the account.

A simple example:

- The current `fee_collectors` of `protocol_instance` are [A].
- `UpdateProtocolInstanceArgs.fee_collectors` = [A, B].
- `UpdateProtocolInstanceArgs.remove_fee_collector` = [A].

`calc_fee_collectors_len` returns a result of 2, so the program reserves space for 2 `fee_collectors` in `protocol_instance`. However, after `protocol_instance::update::processor` completes execution, `protocol_instance` only contains 1 `fee_collector`.

It is recommended to ensure consistency in the operation order of `protocol_instance` between `calc_fee_collectors_len` and `protocol_instance::update::processor` to avoid discrepancies between the allocated space and the actual number of `fee_collectors` in use.



5 Disclaimer

This report reflects the security status of the project as of the date of the audit. It is intended solely for informational purposes and should not be used as investment advice. Despite carrying out a comprehensive review and analysis of the relevant smart contracts, it is important to note that Offside Labs' services do not encompass an exhaustive security assessment. The primary objective of the audit is to identify potential security vulnerabilities to the best of the team's ability; however, this audit does not guarantee that the project is entirely immune to future risks.

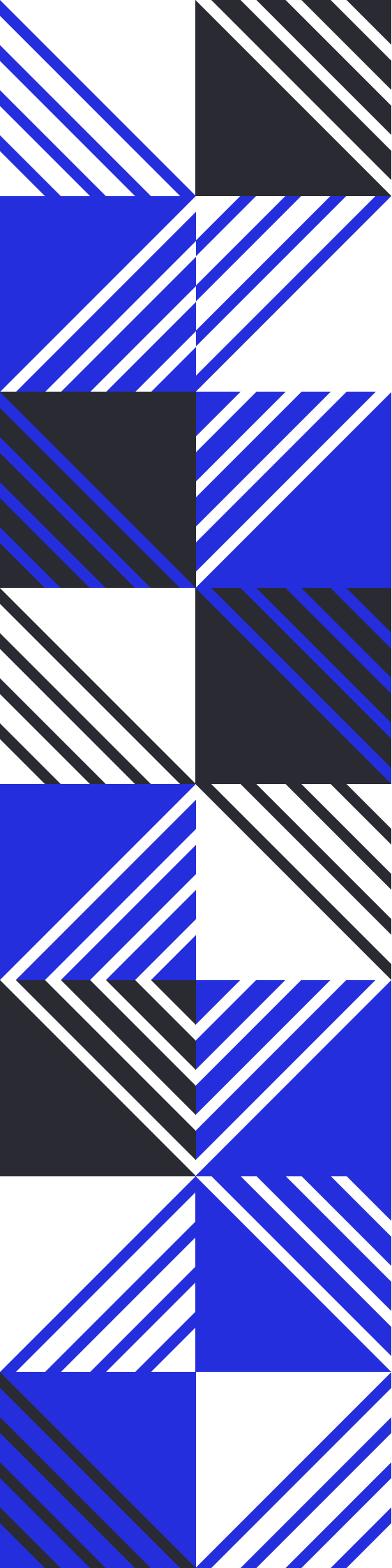
Offside Labs disclaims any liability for losses or damages resulting from the use of this report or from any future security breaches. The team strongly recommends that clients undertake multiple independent audits and implement a public bug bounty program to enhance the security of their smart contracts.

The audit is limited to the specific areas defined in Offside Labs' engagement and does not cover all potential risks or vulnerabilities. Security is an ongoing process, regular audits and monitoring are advised.

Please note: Offside Labs is not responsible for security issues stemming from developer errors or misconfigurations during contract deployment and does not assume liability for centralized governance risks within the project. The team is not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By utilizing this report, the client acknowledges the inherent limitations of the audit process and agrees that the firm shall not be held liable for any incidents that may occur after the completion of this audit.

This report should be considered null and void in case of any alteration.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs